

# Execution-Grounded Mission Feasibility for Intermittent Geohazard Monitoring: Obligation-Level Certificates, a Local Safety Floor, and Lawful Re-Compilation

Pre-disaster monitoring & agentic emergency communication working draft  
agentic-communication repository, method v1.2 — September 19, 2026

## Abstract

We study whether an agent runtime placed *above* the communication entities of a pre-disaster landslide monitoring deployment can keep a monitoring mission executing when field nodes, the LoRaWAN access, and the cellular/short-message backhaul dynamically appear, degrade, disconnect, and recover. Under a source-grounded mountain deployment (battery+solar Class A nodes, a gateway with local autonomy, a 16 h cellular outage, and a sparse BeiDou short-message backup), we first establish a *symmetric feasibility frontier*: with strong reactive baselines (energy-aware sampling, obligation-driven max-coverage backup packing, piecewise mission views, FIFO repair), per-obligation counterfactual relaxations and an omniscient offline packing show that a controller—including a joint optimizer or an LLM—cannot raise on-time delivery by re-optimising collection, scheduling, or packing under the standard edge discipline, without changing resources or the granted authority (the one fixed-resource gain is an orthogonal execution-layer change to record disposition, shown later); the residual failures decompose cleanly into four chain segments. We then show that the genuine, achievable value is *feasibility truth*, not throughput. At mission-compile time, the geometry of the backup schedule alone lets us certify, with 100% recall and zero false positives a median of 7.2 h ahead, the obligations whose delivery window contains no return path; when the mission table finally traverses the outage, the gateway attributes the correct failure segment for 100% of already-missed obligations from its local log. We further find that 45.6% of short-message records delivered during the outage arrive already expired. Critically, feasibility truth also has an *execution-layer payoff*. Under the standard link semantics in which a record is cleared from a node’s FIFO repair cache only once the centre acknowledges it, naively withholding expired backup records is not free: the unacknowledged dead records keep occupying each limited access batch and push fresh samples’ reception past the outage—a cross-layer transfer of cost from the backhaul to the access segment. A *deadline-aware selective purge*, the execution-layer counterpart of the certificate, stops holding and retransmitting only records whose obligation deadline has passed while retaining every still-rescuable record; it is implementable at the node with zero downlink signalling. At the main point it raises outage on-time delivery from 202 to 434 and full-horizon service from 0.400 to 0.432 under unchanged resources, and drives expired sends to zero (send-promise precision  $0.54 \rightarrow 1.00$ ). A paired ten-seed sweep confirms the effect: service rises by a mean 4.21 points (95% CI +3.64 to +4.79, positive on all ten seeds) and outage on-time delivery increases 2.52-fold, while it dominates the AoI latest-only queue by 5.73 points (CI +5.06 to +6.39) on every seed—latest-only matches outage freshness only by discarding still-useful records. EDF and obligation-greedy queues that reorder but never release gain nothing. A deterministic, LLM-free placement study then fixes *where* feasibility must be enforced. A naïve centre that directly complies with the elevated schedule permanently kills 13 of 14 nodes

on every seed (39 deaths over three seeds) and halves service; a centre-side feasibility *envelope* that constrains any planner to lawful actions cuts refused commands by an order of magnitude (2488→222) yet still cannot save the tightest-energy seed (12 deaths), because the night-time relax command must cross the backhaul and then a Class A receive window before it applies. Moving the safety-critical night-energy rule to a *node-local autonomy floor* removes every death (39/12→0) at unchanged service. A purely local upper bound—a pre-provisioned day/night rhythm with zero centre commands—attains the same service (0.360) as every centre-compiled configuration: under static, pre-provisioned, predictable missions a centre controller adds no throughput. The defensible role of the agent runtime is therefore confined to what local autonomy cannot obtain—non-pre-provisioned authorised changes, structural infeasibility disclosure, and renegotiation—mediated by a deterministic envelope that can never override the local safety floor. In a full three-arm live study with a real LLM controller (three outage seeds each for an unaided agent, an agent given an ordinary engineering checklist, and a certificate agent, plus healthy-link controls), the unaided and checklist agents mistake LoRa access for backhaul connectivity, storm ineffective commands, and falsely report the elevated mission as in effect (24 and 19 times), fatally densifying at night (35 and 39 node-deaths); the certificate agent eliminates the false reports and ineffective dense commands, logs structural infeasibility instead, and cuts deaths to 4 (zero on two seeds)—the four tightest-seed residuals that the deterministic local floor then removes—and on a healthy link it does not suppress the feasible upgrade but is strictly better, whereas an unaided agent still kills 13 nodes by over-sampling. Given the certificate, the same agent stops, locates the blocking segment, holds the last lawful configuration, and initiates a standards-authorised re-compilation. The contribution is an execution-grounded obligation model, segment-labelled feasibility certificates, two execution-layer mechanisms (deadline-aware selective purge and a locality-correct local safety floor with a centre feasibility envelope) that turn certified loss into metered-cost savings, higher on-time delivery, and guaranteed survival without adding resources, a symmetric negative result bounding what any centre controller may add under static missions, and lawful re-compilation that aligns what the system promises with what it can actually deliver.

## 1 Introduction

**Where the agent sits.** Prior emergency-communication research combines and recovers *given* network roles and infrastructure: it places UAVs, allocates bandwidth, re-plans topology, or sequences infrastructure recovery tools. We instead place an agent runtime *above* these entities and study its ability to sustain task execution as the entities themselves change state. The setting is pre-disaster monitoring in a mountainous geohazard area, under tight energy and intermittent access/backhaul. The reference deployment keeps field nodes, a gateway with a cache and pre-provisioned autonomous rules, and intermittent backhaul; LoRaWAN Class A is the studied access. Relay and backup backhaul enter only where hardware and standards support them.

**What we expected, and what we found.** An early hypothesis was that an agent that jointly configures collection and backup sending would deliver more than fixed rules. A sequence of increasingly strong, symmetric tests refuted the *throughput* version of that hypothesis, and—importantly—refuted it without rigging the baseline. Under fixed resources, strong reactive policies already sit on the feasible frontier; the failures that remain are physical walls (no return slot, backup capacity, night-time energy), and omniscient offline packing cannot route around them. This bound concerns collection, scheduling, and packing decisions under the standard hold-until-acknowledged edge discipline; we later find one fixed-resource gain orthogonal to it, an execution-layer change to how already-dead records are disposed (§5.6).

Closing the throughput question did not close the research problem. Instrumenting a real LLM

controller exposed a different, sharper failure: the runtime is *blind to where the mission breaks*. During a 16 h backhaul outage it read “nodes are heard over LoRa” as “backhaul is up”, kept emitting the elevated 300 s configuration that the outage made unapplicable (the vast majority of these commands were refused, almost none confirmed), oscillated between configurations every 30 minutes, and reported the elevated mission as running while the field still executed the previous, lower alert level. The same aggregate telemetry (age-of-information, state-of-charge) cannot tell the controller whether data were never sampled, sit at a node, sit at the gateway, or lack a return opportunity, and cannot distinguish a command queued, in transit, or applied.

**The reframed problem.** The controller should compile a changing, authorised monitoring requirement into an *executable plan* over sampling, access, energy, and return opportunities, and—crucially—when a part of the mission cannot be honoured, it must say *which obligation, in which time interval, at which chain segment, is infeasible, and what lawful change would move it*, rather than silently under-delivering or killing nodes to try.

## Contributions.

1. **Obligation-chain model and symmetric feasibility frontier (§4).** A per-obligation chain (qualified sample  $\rightarrow$  gateway  $\rightarrow$  return before deadline  $\rightarrow$  central evidence) and a methodology of symmetric per-obligation counterfactual relaxations. Strong baselines attain the frontier; under fixed resources no controller (rule, joint MPC, or omniscient packing) has a realisable on-time delivery gain. Failures decompose into four segments with zero unexplained remainder.
2. **Execution-grounded feasibility certificates (§5).** Compile-time structural certificates (no return slot; mission/command unreachable) that are 100% sensitive with zero false positives and a 7.2 h median lead, plus arrival-time segment attribution at 100% once the delayed mission table reaches the field. We delimit exactly which claims are deterministic and which require online state.
3. **Lawful re-compilation and real cost (§5,§7).** Re-compilation confined to standards-granted authority (remote frequency adjustment; conference upgrade/downgrade), with measurable cost accounting: 45.6% of outage short-message records arrive expired (a record-level waste upper bound), and certificate-aware holding avoids futile night-time dense sampling that permanently kills nodes.
4. **Execution-layer certificate: deadline-aware selective purge (§5.6,§7).** The same feasibility judgement, applied at the edge, releases from the FIFO repair cache only records proven past their deadline and retains every rescuable one. Under fixed resources it more than doubles outage on-time delivery at the main point (202 $\rightarrow$ 434) and raises full-horizon service 3.2 points there; a paired ten-seed sweep gives a mean +4.21 points (95% CI +3.64 to +4.79, positive on all ten seeds) and 2.52-fold outage on-time delivery, with expired sends cleared and send-promise precision 0.54 $\rightarrow$ 1.00. It is location-independent (a zero-signalling node queue slightly beats a gateway version), dominates AoI latest-only on every seed, and is not reproduced by EDF or obligation-greedy queues.
5. **Execution locality and a layered feasibility runtime (§6).** A deterministic, LLM-free placement study establishes that safety-critical feasibility must be enforced where it can act in time: a node-local night-energy floor and deadline purge (zero downlink), gateway-local autonomy (max-coverage packing, mission-view gating, arrival-time attribution), and a centre runtime

confined to what local autonomy cannot obtain—non-pre-provisioned authorised changes, structural infeasibility disclosure, and renegotiation—with every centre action passed through a deterministic feasibility envelope that cannot override the local floor. The node-local floor removes all permanent deaths (39 and  $12 \rightarrow 0$  over three seeds) at unchanged service; a zero-command purely local upper bound matches every centre-compiled service level (0.360), a symmetric negative result that closes centre throughput/energy control under static pre-provisioned missions; the centre envelope alone cuts refused commands  $2488 \rightarrow 222$  but cannot guarantee survival without the local floor (12 deaths remain on the tightest-energy seed).

6. **Real-agent three-arm evidence (§7.5.)** With identical legal information, adding only the certificate converts a real LLM controller’s segment misdiagnosis, command storm, and silent mis-reporting into segment-correct, lawful, honest behaviour: across three outage seeds the unaided and checklist agents cause 35/39 permanent deaths with 24/19 false upgrade reports while the certificate agent causes 4 (zero on two seeds) with zero false reports, does not suppress a feasible upgrade on a healthy link, and leaves the residual energy-safety guarantee to the deterministic local floor.

We do *not* claim an LLM out-optimises a solver with the full task model; the deterministic projector, packer, and energy ledger are in-layer tools available to rules and agents alike.

## 2 Background, scenario, and standards grounding

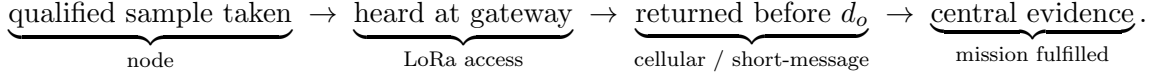
**Deployment.** Fourteen sensor nodes in two groups monitor displacement (and rainfall) on a slope. Nodes sample and upload to a local gateway over LoRaWAN Class A; the gateway forwards to a central server over cellular backhaul, with a BeiDou short-message service as a sparse backup. Nodes are battery plus small solar. The mission is *pre-disaster* and routine, with authorised warning-level changes; the agent never judges landslide risk.

**Standards-grounded authority.** We use two Chinese geological-hazard monitoring industry standards, DZ/T 0460–2023 and DZ/T 0450–2023, retrieved and clause-audited in the repository [1, 2]. Relevant clauses: §5.3.3 permits *remote dynamic adjustment of sampling and upload frequency by warning level* (no numeric levels are given, so the periods we use are declared research choices); §8.4.1 defines four levels (red/orange/yellow/blue) and §8.4.2 permits upgrade, downgrade, or termination after a conference; §5.3.7 calls for dual-mode communication under weak coverage in high-risk extreme weather (a conditional requirement, appearing once); §5.3.6 gives endurance targets. The BeiDou open-service specification (BDS-OS-PS-3.0) [3] reports  $\geq 95\%$  success, average latency under 2 s, an average service frequency of one message per 30 s, and a single-message ceiling of 14,000 bits, while noting that realised frequency and maximum length are constrained by registration parameters—i.e. the rate is a *provisioned* quantity. These clauses bound what the agent may lawfully do: it may re-compile frequency within granted levels and request a conference change, but it may not silently drop an obligation or declare risk itself.

**Why the numbers are what they are.** The short-message service is metered and sparse; the studied operating point uses one 78-byte packet every 1200 s (20 B header, 58 B payload; a displacement sample is 6 B, so roughly 9–10 samples fit per packet). The elevated (yellow) task uses a 300 s sampling period with half-open windows and a delivery deadline two periods after window open, i.e. a  $\approx 600$  s delivery window—*shorter* than the 1200 s backup slot spacing. This geometric mismatch is load-bearing for our structural certificate.

### 3 System model and the obligation chain

Time is slotted at 60 s. A **monitoring obligation**  $o$  is a tuple  $(n, m, [l_o, h_o], d_o)$  over node  $n$ , measurand  $m$ , sampling window  $[l_o, h_o]$  and absolute delivery deadline  $d_o$ . A routine obligation at period  $P$  has  $h_o - l_o = P$  and  $d_o = h_o + P$  (one period of grace). We track a sample through a fixed chain:



An obligation is *fulfilled* only when a sample matching  $(n, m)$  is taken in-window and received centrally by  $d_o$ ; matching uses half-open windows so a boundary sample cannot double-cover adjacent windows.

**Energy and devices.** A sample costs  $4.7 \times 10^{-4}$  Wh; an uplink  $2.33 \times 10^{-5}$  Wh. Battery capacity is 0.05 Wh. Solar input follows a 12 h half-sine with peak 0.03 Wh/h, with per-node/hour stochastic cloud occlusion (35% chance of attenuation to 0.25; expected coefficient  $k_c \approx 0.7375$ ). A node that browns out *permanently* dies in the main configuration (an absorbing state; a restart sensitivity is left open). A cache of 240 slots with FIFO replacement sits at the node. A record is removed from this cache only once it is acknowledged at the centre (the acknowledgement is modelled synchronously and without airtime, a pre-existing simplification that favours every queue discipline equally); until then it is retransmitted oldest-first, with at most 32 records per upload opportunity. Access succeeds with probability 0.74; primary backhaul is good with probability 0.62.

**Class A execution reality.** A configuration command is not immediate: it is queued, transits the backhaul to the gateway, and reaches a node only in its next Class A receive window; it counts as APPLIED only when the node later reports the new config. During a backhaul outage, commands are refused and fresh telemetry stops. A command issued now, applied later, and still consuming energy in between is a first-class effect; dense sampling across a 12 h night needs  $\approx 0.071$  Wh, exceeding the 0.05 Wh battery.

**Mission reachability.** The authorised schedule is piecewise (blue 600 s, then yellow 300 s from  $t=6$  h). Crucially, the *mission table itself* must traverse the backhaul to the gateway; in the main scenario the primary is down from  $t=4$  h to  $t=20$  h, and the table (which does not use the short-message path) reaches the gateway only at  $t=20$  h. Before that instant the field lawfully continues the last APPLIED blue configuration under local autonomy. This is not a modelling convenience: it is the phenomenon we study.

**Strong baselines (the bar to beat).** We assemble deliberately strong ordinary mechanisms, each given the same legal information: **dayfeed**, an energy-aware sampling policy that densifies by day and relaxes through the night (the implicit worst-case-robust reaction); **maxcov**, a textbook max-coverage/recency greedy that packs each backup packet by marginal new obligation coverage per byte; **piecewise mission views** with a gateway gate that withholds future requirements until they actually arrive, while preserving already-open windows across a level change; and **FIFO** access repair. A scripted dayfeed decider driven through the full agent harness reproduces the reference service level bit-for-bit, demonstrating no hidden privilege or penalty for the agent path.

## 4 The symmetric feasibility frontier

Before proposing any mechanism we ask whether controllable, changeable loss exists. We do so with *symmetric* per-obligation counterfactuals: each relaxation widens one physical resource (never the policy), and we report set differences over the same obligation ids, so a single strategy’s good luck cannot masquerade as an upper bound.

### 4.1 Failure-segment attribution

For every unfulfilled obligation we attribute the first blocking segment using an *optimistic* model that grants every assumption favourable to feasibility:

- $S_{\text{time}}$ : even optimistically, no return opportunity (primary slot or backup slot) exists between the qualified sample and  $d_o$ ;
- $S_{\text{energy}}$ : a return opportunity exists but no qualified sample is collected;
- $S_{\text{access}}$ : collected but never heard at the gateway;
- $S_{\text{cap}}$ : collected and heard, but backup capacity / recovery congestion cannot return it by  $d_o$ .

Only the first failing segment is charged. The optimistic model for return opportunities counts deterministic backup slots and an open primary; an obligation is labelled  $S_{\text{time}}$  only when even this favourable model finds no slot. Over the full 48 h, 7560 routine obligations, the strong baseline delivers 3025 (0.400) with zero node deaths; the 4535 failures attribute as  $S_{\text{energy}}$  2002 (26.5%),  $S_{\text{cap}}$  1945 (25.7%),  $S_{\text{time}}$  588 (7.8%),  $S_{\text{access}}$  0.

### 4.2 Relaxation walls

Table 1 relaxes energy (battery 0.05→0.50 Wh with always-dense sampling) and backhaul (1200 s/78 B → 60 s/unlimited) independently and jointly.

| Run      | Sampling/Energy     | Backhaul    | Delivered    |
|----------|---------------------|-------------|--------------|
| base     | dayfeed, 0.05 Wh    | 1200 s/78 B | 3025 (0.400) |
| $O_{bh}$ | dayfeed, 0.05 Wh    | unlimited   | 4740 (0.627) |
| $O_{en}$ | always-300, 0.50 Wh | 1200 s/78 B | 4089 (0.541) |
| $O_{bo}$ | always-300, 0.50 Wh | unlimited   | 6023 (0.797) |

Table 1: Symmetric counterfactuals over 7560 obligations. Set decomposition: 1715 obligations are blocked by backhaul capacity alone, 1128 by battery alone, and only 155 (2.0%) require *both*—and even those need physical augmentation, not coordination.

### 4.3 The sampling/energy subspace is already closed

A feed-forward night-energy policy that ignores two costs kills 13 nodes (service 0.213): it omits the Class A command latency  $L$  (the node keeps sampling densely while the relax command propagates) and the dawn charging ramp  $R$  (solar input is near zero at sunrise). Counting both ( $L=3600$ ,  $R=7200$  s) collapses the policy *bit-for-bit* to dayfeed. A rolling-horizon MPC that integrates the true half-sine harvest gains 1.47 points on seed 0 but averages  $-1.3$  points over six seeds and introduces 21 new permanent deaths on seeds dayfeed survives; a robust sweep over the cloud coefficient has no operating point that adds no deaths and is not worse on average. The reason is structural: legal information does not reveal tomorrow’s clouds, while dense sampling under random

cloud plus an absorbing brownout state has asymmetric cost. Dayfeed’s “relax every night” is an implicit minimax-robust response.

#### 4.4 The outage window: mission unreachability dominates

Focusing on the 2338 yellow obligations whose deadlines fall inside the outage ( $\text{release} \geq 6$  h,  $\text{deadline} \leq 20$  h), Table 2 is decisive. The first failure of an elevated task during an outage is

| Run                                       | collected/heard | delivered | $\Delta$ delivered |
|-------------------------------------------|-----------------|-----------|--------------------|
| base (dayfeed, 0.05 Wh, r1200/78)         | 1181            | 202       | —                  |
| $O_{en}$ (always-300, 0.50 Wh, r1200/78)  | 1181            | 202       | <b>0</b>           |
| $O_{bo}$ (always-300, 0.50 Wh, unlimited) | 1181            | 1120      | <b>+918</b>        |

Table 2: Outage-window obligations ( $n=2338$ ). Removing the energy wall gives *zero* gain: the yellow command cannot traverse the outage, so the field keeps sampling at blue 600 s regardless of battery. Widening the backhaul adds 918. 1218 obligations remain unfulfilled even under both relaxations (no-slot geometry plus access loss).

therefore **mission/command unreachability**, not energy and not packet selection. The field cannot execute a requirement it never receives. An omniscient compile-time EDF packing that assumes every obligation’s sample is present at window start claims 420 obligations fit the backup slots, yet only 9 are realised—the offline packing was optimistic about *supply*, which the unreachable task table prevents. Feasibility projection must take the field’s legally knowable supply rhythm as input, not the nominal mission table.

**Frontier conclusion.** Across the full horizon and within the outage, the two subspaces’ omniscient upper bounds collapse onto the reactive baseline, and the only large gains require adding backhaul capacity or energy. Under fixed resources and authority there is no realisable throughput headroom for a controller, a joint MPC, or an LLM within collection, scheduling, and packing under the fixed edge discipline. (The execution-layer record-disposition action of §5.6 was held constant in these relaxations and is examined separately.) This is a strong, symmetrically backed *negative result* for the control question—and it frees the contribution to live where value genuinely remains.

## 5 Feasibility certificates and lawful re-compilation

We give the runtime an obligation-level feasibility projector that emits three kinds of statement, kept rigorously distinct: **fulfilled** (verified by an independent scorer), **not currently guaranteed** (a knowledge boundary), and **infeasible-under-stated-conditions** (which requires an optimistic model to also fail; a heuristic that merely did not finish may never claim impossibility).

### 5.1 Two placement levels

**Compile-time (central)**, when an authorised change is authored, the centre holds the full obligation table, network-side outage status, and public device/link parameters. It runs an optimistic allocation and emits only *zero-false-positive structural* certificates. **Arrival-time (gateway)**, when a (possibly delayed) mission table arrives, the gateway has the richest local state—heard samples, its own forwarding log, backup slots used—and performs precise per-obligation attribution and

near-horizon packing, while remaining autonomous during outages. Capacity-contention outcomes that depend on online randomness are *not* asserted per-obligation at compile time.

## 5.2 Structural no-return certificate ( $S_{\text{time}}$ )

Backup slots occur at  $t \bmod 1200 = 0$ ; an upgraded obligation’s delivery window is 600 s. An obligation whose window contains no backup slot (and whose primary is in outage) cannot be returned even under optimistic supply and unlimited node energy. This test uses only public geometry and contains no random quantity, so it is seed-invariant.

## 5.3 Mission-unreachable certificate

If the mission table (and hence Class A configuration commands) cannot reach the field before an obligation’s deadline, the field will execute the last applied configuration under local autonomy; the elevated obligation is unreachable as a command, independent of node energy. The centre may derive this from its own command ledger (queued/refused, no APPLIED confirmation) and network-side outage; no future truth is used.

## 5.4 Arrival-time segment attribution

When the delayed table arrives at  $t=20$  h, the gateway replays each already-dead obligation against its local heard/forwarding log and the public slot geometry, assigning  $S_{\text{time}}/S_{\text{cap}}/S_{\text{energy}}/S_{\text{access}}$ .

## 5.5 Lawful re-compilation

For certified obligations the runtime acts only within standards authority: hold the last lawful configuration and local autonomy; suppress futile commands and expired backup sends (the latter only paired with the selective purge of §5.6, never by withholding sends alone); log an obligation-level infeasibility certificate with the segment and the resource change that would move it; after recovery, compile only the guaranteeable subset; and where the mission truly requires more, request added backup quota or a conference downgrade/upgrade under §5.3.3/§8.4.2. The agent never silently marks an unfulfilled obligation complete, never deletes it, and never judges landslide risk.

## 5.6 Execution-layer certificate: deadline-aware selective purge

The compile-time certificate says an obligation cannot be returned; the execution-layer question is what the edge should *do* with records already caught in its caches. Two facts interact. A record leaves a node’s repair cache only on central acknowledgement; otherwise FIFO repair keeps re-sending the oldest unacknowledged records, and during an outage the metered backup path is the only route to such an acknowledgement.

**Why naively suppressing expired sends is not free.** Withholding a backup record that is already past its deadline (the obvious reading of the 45.6% waste) withholds its acknowledgement as well. The record then stays in the node cache and, as an oldest unacknowledged entry, fills the limited access batches, so genuinely fresh samples are not heard until the primary recovers. At the main point this naive suppression saves 29% of backup bytes but costs 8 on-time obligations and 0.25 points of service: cost is moved from the backhaul segment to the access segment, not removed.

**Selective purge.** The execution-layer counterpart of the certificate releases an unacknowledged edge record *if and only if* its obligation deadline has already passed ( $d \leq t$  under the locally known period)—a zero-false-positive test because the deadline is absolute—while retaining every record with  $d > t$ . This both stops the expired backup send and frees the access batch. It has two equivalent placements: a node-local queue discipline that needs no downlink at all (**deadline-purge**), and a gateway action that locally acknowledges records it will not send (**cert-purge**); both use only the public period  $P$  legally available before the mission table arrives. The two placements implement one deadline judgement and give the same result. Dropping stale or expired packets is itself well known in AoI packet management and DTN bundle expiry [4, 5]; the novelty is not the discard but its obligation-feasibility trigger and the requirement that it be consistent across both segments (see §8).

**Why ordinary queues do not suffice.** EDF and obligation-greedy disciplines reorder which records occupy a batch but never release them, so the cache is not freed and we measure no gain. The AoI latest-only queue does release records, but indiscriminately (it keeps only the newest), matching outage freshness while permanently abandoning records still deliverable after recovery and lowering full-horizon service. The certificate supplies the *selectivity* that turns a blunt freshness heuristic into a service-preserving action; this is an execution-layer mechanism a fixed rule realises, and whose per-obligation judgements an agent must maintain when requirements change.

**Relationship to the feasibility frontier.** The negative result of §4 is scoped to collection, access scheduling, and backhaul packing under the standard edge discipline that *holds every unacknowledged record until the centre confirms it*. Selective purge is an orthogonal execution-layer action: it changes a record’s *lifetime disposition*, not resource use, and that disposition was held fixed in every counterfactual relaxation. Its gain is therefore not throughput extracted from a physical wall; it removes an internal leak in which records already proven unrecoverable keep consuming access batches. The symmetric frontier bounds what better *decisions* achieve given the protocol’s state handling; the execution-layer certificate corrects that state handling itself.

## 6 Execution locality and the layered runtime

The selective-purge result already hints at a placement principle: the zero-downlink node-local queue slightly beats the gateway version, because it acts without waiting for any round trip. We now make that principle explicit and test it deterministically (no LLM, three seeds, the main operating point). The question is not how smart a centre planner is, but *where a feasibility constraint can be enforced in time*.

**Two deterministic mechanisms.** The first is a centre-side feasibility **envelope**, a pure filter wrapping any planner (rule, MPC, or LLM) and using only signals the planner itself may legally see (each node’s last *confirmed* configuration, how many nodes were heard in the recent window, the authorised requirement, and the local clock). It enforces: (E1) a night-energy hard gate—during 18:00–06:00 every target is clamped to sparse and nodes currently dense are explicitly downgraded, overriding both mission level and any apparent “recovery”, with 16:00–18:00 closed to new dense sessions; (E2) a control-plane confirmation gate—inside the 06:00–16:00 compile window, a node not yet confirmed dense is not upgraded while fewer than all nodes are heard, since the command cannot currently apply; and (E3) sparse under the blue level. The second mechanism is a **node-local energy floor**: on every 60 s tick the node itself, on its local clock, clamps sampling/report

to sparse at night and self-downgrades at dusk, with no centre command at all; an optional pre-provisioned schedule additionally lets the node execute the authorised day/night rhythm locally. This models the gateway/node “pre-provisioned autonomous rules” already in the deployment; it is off by default and all pre-existing tests remain bit-identical when off.

**Why a centre envelope cannot guarantee survival.** A dense 300 s night costs  $\approx 0.071$  Wh, above the 0.05 Wh battery, and a brownout is absorbing (permanent death). A centre-issued dusk downgrade must traverse the cellular backhaul, then wait for a Class A receive window; under the outage and geometric loss it does not arrive before dusk on the tightest-energy seed. The same rule—“never stay dense overnight”—is a late, refusable *suggestion* at the centre but an invariant enforced every tick *at the node*. Table 3 shows the consequence: naïve centre compliance kills 13/14 nodes on every seed; the centre envelope removes the command storm but 12 nodes still die on the tightest seed; the node-local floor removes every death on every seed without lowering service.

| Configuration (centre placement unless noted)      | Mean svc     | Deaths (3 seeds) | Cmds sent | Refused    |
|----------------------------------------------------|--------------|------------------|-----------|------------|
| comply (naïve, no protection)                      | 0.162        | 39               | 160       | 2488       |
| dayfeed-c (centre day/night rule)                  | 0.370        | 12               | 308       | 1256       |
| envelope(comply), centre only                      | 0.359        | 12               | 308       | 298        |
| comply + local floor                               | 0.366        | 0                | 1424      | 3364       |
| dayfeed-c + local floor                            | 0.368        | 0                | 258       | 1156       |
| <b>envelope(comply) + local floor</b>              | 0.359        | <b>0</b>         | 282       | <b>222</b> |
| local-dayfeed (existing conservative local policy) | 0.296        | 0                | 0         | 0          |
| <b>pure-local (pre-provisioned, 0 centre cmd)</b>  | <b>0.360</b> | <b>0</b>         | 0         | 0          |

Table 3: Deterministic three-seed placement study (no LLM). Centre rows share one policy harness, centre placement, and the queued/in-transit/applied command path; only the decider and the local floor differ. The local floor removes all permanent deaths at unchanged service; the envelope removes the command storm (refused 2488→222) but cannot save the tightest seed without the floor. *pure-local*—a node-local pre-provisioned day/night rhythm with zero centre commands—matches every centre-compiled service level, closing centre throughput/energy control under static pre-provisioned missions. The slightly higher unprotected dayfeed-c mean is bought by 12 deaths (dense sampling before nodes expire) and is not a survivable operating point.

**A symmetric local upper bound closes the static control question.** *pure-local* attains mean service 0.360 (range 0.317–0.387) with zero deaths and zero centre commands, statistically indistinguishable from every centre-compiled configuration (0.359–0.368); a run in which the centre still issues comply commands while the node executes the pre-provisioned rhythm gives bit-identical service but 1274 sent / 2354 refused commands—the centre’s sampling commands are pure overhead under a static, predictable mission. Local autonomy collects more (fewer missing collections, 2016 vs.  $\approx 2650$ ) but returns no more (more missing deliveries), confirming the backhaul, not collection policy, as the binding wall. This is a symmetric, upper-bound-backed closure of the same subspace the omniscient packer closed in §4, now from the placement direction.

**The layered runtime.** The results prescribe a three-layer feasibility runtime, each holding only the constraints it can act on in time: **L0, node-local (zero-signalling safety floor)**—the night-energy floor and obligation deadline-purge, guaranteeing no irreversible death and no dead-record

cache exhaustion; **L1, gateway-local (field autonomy)**—max-coverage backup packing, the non-prescient mission-view gate, arrival-time segment attribution, and autonomous operation during outages; and **L2, centre agent runtime**—only what local autonomy cannot obtain: cross-layer compilation and lawful re-compilation/renegotiation of *non-pre-provisioned* authorised changes, compile-time structural infeasibility certificates with the three-way statement discipline, and cross-segment-consistent disposition and honest external reporting. Every L2 action passes through the deterministic envelope and can never override L0/L1 safety. Rules, MPC, and solvers benefit from the same layers: this is a general communication execution layer, not a claim that an LLM is irreplaceable. Under static, pre-provisioned, predictable missions L2 rightly stays silent; its value appears when requirements change and the system must promise only what it can execute and say, early and precisely, why it cannot do the rest.

## 7 Evaluation

**Setup.** Seed 0 main point (with multi-seed checks where randomness enters): 48 h + 1 h tail, 14 nodes, two groups, cap 0.05 Wh, solar peak 0.03 Wh/h, outage 4–20 h, backup 1200 s/78 B with maxcov, blue→yellow (600→300 s) at 6 h, mission table reaching the gateway at 20 h. All numbers are produced by the repository scripts (r29–r39), each run reproduces 18/18 repository checks and 5/5 joint invariants; the local safety floor is off by default and the disabled-backup and no-floor configurations remain bit-identical to the v1.1 anchors. The placement study of RQ4 is deterministic (no LLM) over seeds 0–2; node death is a deterministic/highly robust quantity, while mean-service differences below one point are not claimed significant.

### 7.1 RQ1: Do structural certificates provide early, precise warning?

**Yes, and deterministically.** At compile time (6 h) the geometry test emits 602 no-slot certificates over the outage window; 588 obligations truly end as  $S_{\text{time}}$  (100% recall of the 588), with 14 boundary cases at the recovery edge attributed to  $S_{\text{cap}}$ , and **zero false positives** (no certified obligation is delivered). Lead time  $d_o$ —6 h ranges 0.2–14 h, median 7.2 h. By contrast, an optimistic-energy capacity certificate is impure (552 correct  $S_{\text{cap}}$ , 571 actually  $S_{\text{energy}}$ , 193 false positives), which is precisely why we restrict compile-time claims to structural/unreachable segments and push capacity attribution to the gateway.

### 7.2 RQ2: Can the field attribute failures when the table finally arrives?

**Yes, at 100%.** 2136 yellow obligations are already past deadline when their table arrives at 20 h. From its local complete log the gateway attributes the correct segment for 2136/2136 (true segments:  $S_{\text{time}}$  588,  $S_{\text{cap}}$  977,  $S_{\text{energy}}$  571). Deterministic *advance* certification before the deadline is essentially impossible at the field (4 marginal cases), because the field physically does not know these obligations exist—the certificate value at this level is immediate, segment-labelled accounting versus silent, un-explained loss.

### 7.3 RQ3: Is real cost wasted on the infeasible, and can a certificate recover it?

During the outage 75 backup packets carry 750 records (5848 B). Of 480 records actually received centrally via backup during the outage, 261 are on-time and **219 (45.6%) are already expired** (1270 of 2782 payload bytes). The tempting response—simply stop sending expired records—is not free, for the acknowledgement reason of §5.6. Table 4 holds the strong maxcov backup chooser

fixed and varies only the edge queue/action. Naive suppression removes the 219 expired sends but loses 8 on-time obligations (202→194) and 0.25 service points: tracing at record level, nine fresh samples taken during the outage are, because the unacknowledged dead records fill each access batch, not heard until after the primary recovers. Completing the certificate at the edge reverses the sign. The node-local **deadline-purge**, which needs no downlink, raises outage on-time obligations 202→434 and full-horizon service 0.400→0.432, clears expired sends, and raises send-promise precision 0.54→1.00; the gateway **cert-purge** is within 0.2 points, confirming that the gain is the deadline judgement rather than its location. Reordering alone (EDF, obligation-greedy) frees no cache and gains nothing; the AoI latest-only queue reaches the outage figure but abandons records still deliverable after recovery, collapsing full-horizon service to 0.363. **Paired**

| Edge queue / action                   | Full svc     | Outage on-time | Expired sends | Send prec.  |
|---------------------------------------|--------------|----------------|---------------|-------------|
| maxcov + FIFO (baseline)              | 0.400        | 202            | 219           | 0.54        |
| maxcov-ontime: suppress, no purge     | 0.398        | 194            | 0             | 1.00        |
| maxcov + obligation-greedy            | 0.392        | 205            | 216           | 0.55        |
| maxcov + EDF                          | 0.400        | 202            | 219           | 0.54        |
| maxcov + latest-only (AoI)            | 0.363        | 431            | 0             | 1.00        |
| <b>maxcov + deadline-purge (node)</b> | <b>0.432</b> | <b>434</b>     | <b>0</b>      | <b>1.00</b> |
| cert-purge (gateway) + FIFO           | 0.430        | 420            | 0             | 1.00        |

Table 4: Seed 0, strong maxcov backup chooser fixed; only the edge queue/action changes. Outage on-time counts upgraded obligations delivered by deadline; expired sends are backup records received centrally after their deadline; send precision is the share of backup sends that were on-time. Naive suppression clears expired sends but loses 8 on-time obligations (the cross-layer FIFO cost of §5.6); deadline-aware selective purge clears them *and* adds 232 on-time obligations under unchanged resources. Reordering queues do not help; AoI latest-only trades post-recovery coverage for outage freshness.

**ten-seed sweep.** Across seeds 0–9 (the maxcov chooser fixed, only the edge queue changes), FIFO→deadline-purge raises mean full-horizon service from 0.373 to 0.415—a paired mean gain of 4.21 points (sd 0.81, 95% CI +3.64 to +4.79 using  $t_{0.975,9}=2.262$ ), positive on all ten seeds (range +3.2 to +5.6)—and raises total outage on-time delivery 1691→4265 (2.52×), while expired backup records fall from 2538 to 1. The three seeds never touched during development (6/7/8) fall in the same range (+5.6/+4.4/+3.7). Deadline-purge also dominates the AoI latest-only queue on *every* seed, by a mean 5.73 points (95% CI +5.06 to +6.39): latest-only matches outage freshness but abandons post-recovery coverage. The energy analogue is larger: always-dense compliance kills 13 nodes, whereas certificate-aware holding keeps deaths at zero.

#### 7.4 RQ4: Does enforcement placement determine safety and command hygiene?

**Yes, deterministically.** Table 3 (mechanisms in §6) yields three findings. (i) Letting the centre directly drive sampling is harmful, not merely suboptimal: naïve compliance permanently kills 13/14 nodes on all three seeds (39 total) and halves service to 0.162. (ii) A centre-side envelope alone is insufficient for safety: it removes the command storm (refused 2488→298) and protects two seeds, but on the tightest-energy seed 12 nodes still die, because the dusk downgrade cannot cross the backhaul and a Class A window in time. The node-local floor removes all 39/12 deaths at unchanged service—the safety-critical constraint must sit where it can act before the energy is spent. (iii) The envelope’s own payoff is command hygiene: with the floor present, adding the envelope cuts

refused centre commands from 3364 (naïve) or 1156 (centre dayfeed) to 222 at the same zero-death service level—a real saving when backhaul and Class A downlink opportunities and short-message billing are scarce. A purely local, zero-command pre-provisioned rhythm reaches 0.360 service, matching every centre-compiled arm; under static, pre-provisioned missions the centre’s sampling commands add neither service nor safety, only cost. The centre’s genuine remit is therefore the non-pre-provisioned and the declarational, examined next.

## 7.5 RQ5: Does the certificate change a real agent?

We first replay three genuine outage-window decision points from an unaided deepseek-flash controller trace with identical system prompt, legal observation, and history; the A1 condition appends only a certificate derived from the agent’s own command ledger and public parameters (no future truth). Table 5. The certificate does not optimise for the agent; it integrates signals the agent al-

| Time  | Unaided A0 (original live trace)                                                                   | A1 + certificate                                                                                                                                        |
|-------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 08h00 | Sets all 14 nodes to 300/300, notes “backhaul up, link healthy” (LoRa heard mistaken for backhaul) | <code>actions: []</code> — outage since 4h, yellow cannot compile, hold blue 600, log infeasibility                                                     |
| 14h30 | Sets all to 300 (“soc ok”), 30 min after having set 600 for the night; oscillating                 | <code>actions: []</code> — outage persists, yellow unreachable and lethal overnight; hold blue, cite §5.3.3/§8.4.2                                      |
| 19h30 | Sets all to 300 at deepest night (cannot apply; would drain nodes)                                 | <code>actions: []</code> — outage blocks Class A; hold last applied blue; log unreachable/no-slot/energy; recompile guaranteeable subset after recovery |

Table 5: Within-subject real-LLM witness. A0 misreads the chain segment, emits ineffective commands that oscillate, and implicitly reports yellow as in effect; A1 is segment-correct, lawful, and honest at all three points. This is an existence proof ( $n=3$ , one model), not an effect-size estimate.

ready received but could not combine (confirmed-applied configs, queued commands, slot geometry, night energy), turning a scalar “age / battery” view into a chain-segment view.

**Full three-arm live sweep.** Beyond the within-subject replay, we ran the complete doc38 protocol against live deepseek-flash: three seeds  $\times$  three arms over the 16-hour outage, plus two healthy-link negative controls (A0 and A1), each a full 48 h+tail run with a real model call at every 30-minute decision (99/99 calls, no heuristic fallback; Table 6). All arms share the identical harness, legal observation, tools, history, and action space; only the decider’s information block differs. The preregistered hypotheses concern survival, honest reporting, command hygiene, and non-suppression on a healthy link; death and reporting counts are robust, whereas per-run service is noisy at  $n = 3$ .

*Survival (H4):* A0 and A0-structured cause 35 and 39 permanent node-deaths across the three seeds (the checklist kills 13 on every seed); A1 cuts this to 4—zero on seeds 0 and 2, four residual on the tightest seed 1. *Honest reporting (H2):* over outage windows A0/A0-structured emit 24/19 false “upgrade in effect” confirmations and 53/42 dense commands that never take effect, whereas A1 emits zero of either, holds sparse sampling at every outage decision (hold fraction 1.0), and writes 89 structural-infeasibility entries, reporting what cannot be guaranteed instead of over-promising. *Command hygiene (H1):* A1 issues 342 commands with 182 refusals (0.53 refusals per sent), against 500/2008 (4.0) for A0 and 496/1420 (2.9) for A0-structured. *Service:* A1 attains the highest mean

| Arm              | svc (s0/s1/s2) | svc mean | dead (s0/s1/s2) | false | ineff. dense | infeas. log | sent/ref |
|------------------|----------------|----------|-----------------|-------|--------------|-------------|----------|
| A0               | .160/.319/.408 | .296     | 13/13/9 (35)    | 24    | 53           | 35          | 500/2008 |
| A0-structured    | .181/.221/.265 | .222     | 13/13/13 (39)   | 19    | 42           | 32          | 496/1420 |
| A1 + certificate | .381/.316/.377 | .358     | 0/4/0 (4)       | 0     | 0            | 89          | 342/182  |

Table 6: Live three-arm runs over the 16-hour outage, three seeds, 99 real model decisions per trace with no heuristic fallback. Service per seed and mean; permanent deaths per seed (total in parentheses); outage-window counts totaled over seeds: false “upgrade in effect” reports, ineffective dense commands, structural-infeasibility log entries, and commands sent/refused.

service (.358), equal to the deterministic safe floor (.360, Table 3); the seed-1 service values overlap (.316 vs .319), so we draw no significance from  $n = 3$  service means—the death, reporting, and command counts carry the result. The ordinary checklist does not close the gap (A0-structured is no better than A0, and worse on mean service and total deaths), attributing the gain to the feasibility certificate rather than to prompt engineering or bookkeeping.

*Healthy-link negative control (H3).* With no outage the certificate must not suppress a feasible upgrade. A1 attains service .511 with zero deaths and zero false/infeasible reports, whereas unaided A0 reaches only .275 and still kills 13 nodes: it over-samples through the elevated segment even with a healthy backhaul, so its energy failure is self-inflicted rather than outage-induced. The certificate is thus not trading service for caution where the network is healthy.

The four residual seed-1 deaths show that the L2 certificate—a per-decision advisory to a live model—does not itself guarantee energy safety in the tightest realization; the deterministic L0 node-local energy gate (§6, off by default and bit-identical when disabled) removes exactly those deaths at unchanged service. We report the live runs as real-agent existence and directional evidence with one model and three seeds; the mechanistic and safety guarantees rest on the bit-identical, seed-swept deterministic RQ3/RQ4 results, not on the LLM.

## 8 Related work

**LLMs in emergency / network control.** TopoLLM [6] uses an LLM for post-disaster rescue topology and resource planning; subsequent lines let LLMs (i) directly schedule UAV path/data collection and bandwidth, (ii) shape rewards for deep RL, or (iii) orchestrate infrastructure-recovery tools (e.g. graph-guided candidate-plan repair for network restoration and LLM-driven operation of real network-management APIs) [7, 8]. These decide *network resources* or compose recovery tools; they do not maintain a long-horizon routine monitoring workflow across collection, configuration, return, and recovery under intermittent links and tight energy, and they do not produce per-obligation feasibility certificates grounded in the execution chain. Intent/graph-based restoration systems already include intent objects, budget/time constraints, candidate-plan checks and local repair, and LLM systems already orchestrate documented APIs [7, 8]; we therefore do not claim “natural language to constraints” as novel. Our delta is the per-obligation chain projection, the segment-labelled infeasibility certificate with zero-false-positive structural tests, and the compile-time/arrival-time split under mission unreachability.

**Scheduling, freshness, energy.** Constrained-queue scheduling with exogenous arrivals and finite service opportunities provides our formal anchors [9]; age-of-information and energy-harvesting link optimisation—including instantaneous-battery threshold policies and partially-observed energy-harvesting control [10, 11]—and overbooking/scheduled-aware forwarding for space links [12] address pieces of the resource problem. We use these as strong baselines and upper-bound tools, not

as strawmen, and we do not claim joint sampling/transmission/energy optimisation as novel. Our setting differs in remote *persistent* configuration with Class A apply latency and in the separated access vs. primary/backup return process, and—decisively—in asking what the controller should *say and lawfully do* when the mission is structurally unfulfillable, rather than only how to pack more bits. Consistent network update work has long studied per-update validity, dependencies, and loop-freedom during transitions [13]; we rely on that lesson (versioning, dependencies, local repair are not new) and place task-transition effects after it. Network calculus and delay/disruption-tolerant triage under extreme contacts are complementary deterministic characterisations of the walls we measure [14].

**Packet discard, freshness, and DTN expiry.** Discarding stale or expired packets is itself well established: AoI packet management pre-empts or drops old updates to lower the receiver’s age [4, 15, 16]; deadline-aware buffer discard has been coupled with adaptive ARQ for video [17]; a sensor transport protocol suppresses packet generation when the estimated delay exceeds the time limit [18]; DTN bundle protocols attach a lifetime and expire bundles hop by hop, alongside a family of TTL/encounter-based drop policies [5, 19]; and LoRa–satellite rural systems use priority queues and de-duplication [20]. These mechanisms decide from packet age, a local buffer deadline, a single packet’s estimated delay, or a bundle TTL, and the discard takes effect locally at one hop. Our selective purge differs in three coupled respects: (i) the release test is the *obligation’s* end-to-end feasibility—its absolute delivery deadline against legally known return geometry—not packet age or a local TTL; (ii) suppression must be *consistent across the access and backhaul segments*, because withholding an expired backhaul send while leaving the record in the node’s acknowledgement-driven FIFO repair cache back-pressures fresh access traffic and costs on-time obligations (§5.6), a cross-segment interaction absent from single-hop AoI discard and from hop-local bundle expiry; and (iii) release is paired with retaining every still-rescuable record, which AoI latest-only (whose objective ignores coverage) does not do. DTN custody transfer [21] likewise moves retention and retransmission responsibility, but as a *forward* hand-off: a downstream custodian accepts a bundle and releases the upstream custodian, explicitly to decouple reliability from any end-to-end acknowledgement; it is keyed to bundle custody rather than to a specific obligation’s deadline, has no notion of two segments coupled by a link-layer repair cache, and has been migrated out of the core Bundle Protocol in BPv7 [22]. We claim not the act of dropping expired packets but the obligation-feasibility test, its cross-segment consistency, and its identity with the compile-time certificate.

## 9 Discussion, limitations, and threats to validity

**Scope of claims.** The throughput negative result is established at one main operating point with symmetric relaxations and strong (including omniscient and MPC) baselines; the structural  $S_{\text{time}}$  certificate is geometric and seed-invariant, but capacity and night-energy *forecasts* are deliberately not claimed as deterministic—they depend on online state and unknown weather. The real-LLM evidence uses one model with three outage seeds per arm (plus two healthy-link controls), each a single temperature-0 trajectory, and per-run service variance is large at  $n = 3$ ; the live sweep shows robust death and reporting counts (unaided A0/A0-structured cause 35/39 deaths with 24/19 false upgrade reports, the certificate agent 4 deaths with zero false reports) but is not a powered service-significance test, and it motivates the deterministic layer—the L2 certificate leaves four residual deaths on the tightest seed, and the L0 node-local gate, not the L2 advisory, provides the energy-safety guarantee. Effect-size and the safety guarantee rest on the deterministic three-seed placement study (RQ4), which itself spans one main operating point; wider deployment

ranges (backup rate/payload, node count, capacity, cloud cover, brownout restart threshold) are not swept, and sub-point mean-service differences are not treated as significant. The static-mission closure assumes a pre-provisioned or predictable schedule. A full three-placement comparison for *runtime, non-pre-provisioned* warning-level changes—centre long loop versus gateway-local LoRa re-compilation versus node pre-provisioning—including in-flight cross-generation commands and the residual physical effects of superseded configurations, is left as future work; the unreachable half is already quantified (RQ2), the reachable half converges to the local operating point once the table arrives (§6), and consistent-update/versioning mechanisms are established in the literature, so we claim no novelty there. The brownout model is absorbing; a restart sensitivity could move the energy wall but not the slot/capacity walls. The centre’s knowledge of outage duration assumes network-side operations visibility; we also give a conservative variant that assumes only the current outage and counts deterministic backup slots. Standards text for one channel was obtained from a document-sharing host (top-grade document, lower-grade copy chain) with the official host unreachable by TLS from our environment; clause numbers were cross-checked from two independent copies. We did not execute authors’ reference implementations and therefore do not claim replication of any cited system; one field study endpoint was rate-limited from our network and is cited without its numbers.

**Selective-purge assumptions.** (i) The central acknowledgement is modelled synchronously and without airtime, a pre-existing v1.1 simplification that favours every queue discipline equally; the node-local **deadline-purge** needs no downlink at all and is our primary mechanism, whereas the gateway **cert-purge**’s explicit acknowledgements would consume Class A receive windows in a real deployment. (ii) A purged record forgoes any retrospective value after its deadline; where such value matters it can be retained at the gateway rather than sent. (iii) During the outage the node and gateway test deadlines against the public pre-change period  $P=600$ ; the exact post-upgrade  $P=300$  deadline enters only post-hoc accounting, never as an online input. (iv) Edge-queue results rest on a paired seed 0–9 sweep with 95% confidence intervals (all service gains positive); the purge test is deterministic given the public period, so residual seed dependence enters through access/solar randomness, and wider deployment parameter ranges are not swept.

**Generality.** The obligation chain and the three-way statement discipline transfer to any intermittent, energy-limited, authority-bounded monitoring workflow; the specific walls are parameterised by the deployment and should be re-derived per deployment.

## 10 Conclusion

For pre-disaster geohazard monitoring under intermittent connectivity, strong ordinary controllers already extract essentially all on-time delivery that fixed resources allow; the remaining failures are physical and cleanly attributable. The defensible role for an agent runtime is therefore neither to out-pack the network nor to directly drive safety-critical sampling, but to sit *above* a layered, execution-grounded substrate. Node-local and gateway-local autonomy enforce, in time and with zero or local signalling, the night-energy floor and record disposition that no centre round trip can guarantee: a deterministic placement study shows that unprotected centre compliance permanently kills nodes, that a centre-side feasibility filter alone cannot save the tightest-energy seed, that a node-local floor guarantees survival at unchanged service, and that a purely local pre-provisioned rhythm already matches every centre-compiled service level under static missions. Above that floor the centre maintains *execution-grounded honesty*: it compiles authorised missions against the real collection/access/return/energy chain, certifies structural infeasibility hours ahead with no false alarms, attributes every miss to its segment the moment evidence arrives, and—at the execution

layer—releases exactly the records a certificate proves unrecoverable while keeping every rescuable one, so that honesty about infeasibility also frees real on-time delivery rather than merely moving cost between the access and backhaul segments; it then re-compiles or re-negotiates only within the law. The agent’s value is thereby reserved for what local autonomy cannot have—changing, non-pre-provisioned requirements and honest, early, segment-precise disclosure—so that the system promises exactly what it can execute, and says, precisely and early, why it cannot do the rest.

## References

- [1] Ministry of Natural Resources of the People’s Republic of China. DZ/T 0460–2023, technical specification for geological-disaster monitoring (clauses 5.3.3, 5.3.6, 5.3.7, 8.4.1, 8.4.2). Technical report, Industry standard of the People’s Republic of China, 2023.
- [2] Ministry of Natural Resources of the People’s Republic of China. DZ/T 0450–2023, communication-layer specification for geological-disaster monitoring (clauses 7.4.2.3, 7.4.3.2, 7.4.3.5). Technical report, Industry standard of the People’s Republic of China, 2023.
- [3] China Satellite Navigation Office. BDS-OS-PS-3.0, beidou navigation satellite system open-service short-message communication service specification (table 7-1). Technical report, China Satellite Navigation Office, 2023.
- [4] Maice Costa, Marian Codreanu, and Anthony Ephremides. On the age of information in status update systems with packet management. *IEEE Transactions on Information Theory*, 62(4):1897–1910, 2016. Conference version: Age of information with packet management, IEEE ISIT 2014, pp. 1583–1587.
- [5] Kevin Fall, Scott Burleigh, Adrian Hooke, et al. Delay-tolerant networking architecture (bundle layer, bundle lifetime/expiration). IRTF RFC 4838, 2007.
- [6] TopoLLM: Large language model based topology and resource planning for post-disaster rescue networks. *Digital Communications and Networks*, 12 (2026) 273–282, online 2025-10-17.
- [7] ICG-Restore: intent- and graph-guided candidate-plan checking and local repair for infrastructure restoration. *AI*, 7(8):294, MDPI, 2026.
- [8] OSS-GPT: orchestrating real network-management operations from interface documentation. EureCom technical publication, ComSys publi-8206.
- [9] Constrained-queue scheduling with exogenous arrivals and finite service opportunities for iot data collection (ICLDC). arXiv:2504.14556; also *IEEE Internet of Things Journal*, DOI 10.1109/JIOT.2025.3615410, 2025.
- [10] Age-optimal policies with energy harvesting and instantaneous battery state. arXiv:1905.06679, 2019.
- [11] Energy harvesting wireless control under partial observability (EH-POMDP). arXiv:2311.06522, 2023.
- [12] CCSDS 734.3-b-1, scheduled-aware bundle routing and overbooking for delay-tolerant space links. Consultative Committee for Space Data Systems, blue book.

- [13] On dependencies, loop freedom, and blackholes in consistent network updates. Microsoft Research technical report.
- [14] Jean-Yves Le Boudec and Patrick Thiran. *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*. Springer, LNCS 2050, 2001.
- [15] Chih ping Kam, Sennur Kaya, and Anthony Ephremides. Controlling the age of information: Buffer size, deadline, and packet replacement. In *Proc. IEEE Military Communications Conference (MILCOM)*, 2016.
- [16] Ashok Krishnan K. S. and Vinod Sharma. Minimizing age of information in a multihop wireless network. arXiv:1909.06571, 2019. Distributed scheduling plus a local drop rule that keeps only the newest packet of each flow at each queue (no inter-node signalling); the keep-latest multihop baseline.
- [17] Rouzbeh Razavi, Martin Fleury, and Mohammed Ghanbari. Deadline-aware video delivery in a disrupted Bluetooth network. In *Proc. IEEE Sarnoff Symposium*, 2007. IEEE Xplore document 4567352; deadline-aware active discard from the send buffer co-located with picture-type adaptive ARQ on a single Bluetooth piconet (verified from publisher/abstract-level record; IEEE full text not opened in this environment).
- [18] Takahiro Wada, Hayato Kitamoto, Takerou Fukuhara, and Iwao Sasase. A transport protocol with energy consumption reduction by reducing redundant transmissions in wireless sensor and actor networks. In *Proc. IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM)*, 2009. IEEE Xplore document 5291354; a source node does not generate a packet when its estimated one-hop-times-hops delay already exceeds the remaining time limit.
- [19] Saeid Iranmanesh. A novel queue management policy for delay-tolerant networks. *EURASIP Journal on Wireless Communications and Networking*, 2016(88):1–23, 2016. QM-EBRP: local forward/drop utility from time-to-live, encounter rate and replica count under quota-based routing.
- [20] SateRloT: high-performance ground-space networking for rural IoT (LoRa and LEO satellite, priority queue and de-duplication). ACM MobiCom, 2024.
- [21] Kevin Scott and Scott Burleigh. Bundle protocol specification. Technical Report RFC 5050, IRTF / RFC Editor, 2007. Custody transfer: on accepting custody a downstream node commits to retain/re-forward a bundle and releases the upstream custodian; the mechanism is a forward hand-off that decouples retransmission responsibility from end-to-end acknowledgement.
- [22] Bundle protocol version 7. Technical Report RFC 9171, IETF / RFC Editor, 2022. Appendix A: custody transfer is migrated out of the core bundle protocol to bundle-in-bundle encapsulation (BIBE).